

Continuous motion recognition

In this tutorial, you'll use machine learning to build a gesture recognition system that runs on a microcontroller. This is a hard task to solve using rule based programming, as people don't perform gestures in the exact same way every time. But machine learning can handle these variations with ease. You'll learn how to collect high-frequency data from real sensors, use signal processing to clean up data, build a neural network classifier, and how to deploy your model back to a device. At the end of this tutorial you'll have a firm understanding of applying machine learning in embedded devices using Edge Impulse.

There is also a video version of this tutorial:

Building a Continuous Motion Recognition System with Embedded Machine ...



You can view the finished project, including all data, signal processing and machine learning blocks here: [Tutorial: continuous motion recognition](#).

1. Prerequisites

For this tutorial you'll need a supported device. Follow the steps to connect your development board to Edge Impulse.

- [ST B-L475E-IOT01A](#)
- [Arduino Nano 33 BLE Sense](#)

- [Silicon Labs Thunderboard Sense 2](#)
- [Any mobile phone](#)

If your device is connected under **Devices** in the studio you can proceed:

Your devices
These are devices that are connected to the [Edge Impulse remote management API](#), or have posted data to the [ingestion SDK](#).

NAME	ID	TYPE	REMOTE MANAGEMENT	LAST SEEN	
 Jan's DISCO-L475VG	C4:7F:51:8C:8E:CC	DISCO_L475VG_IOT01A		Today, 12:26:37	

Devices tab with the device connected to the remote management interface.

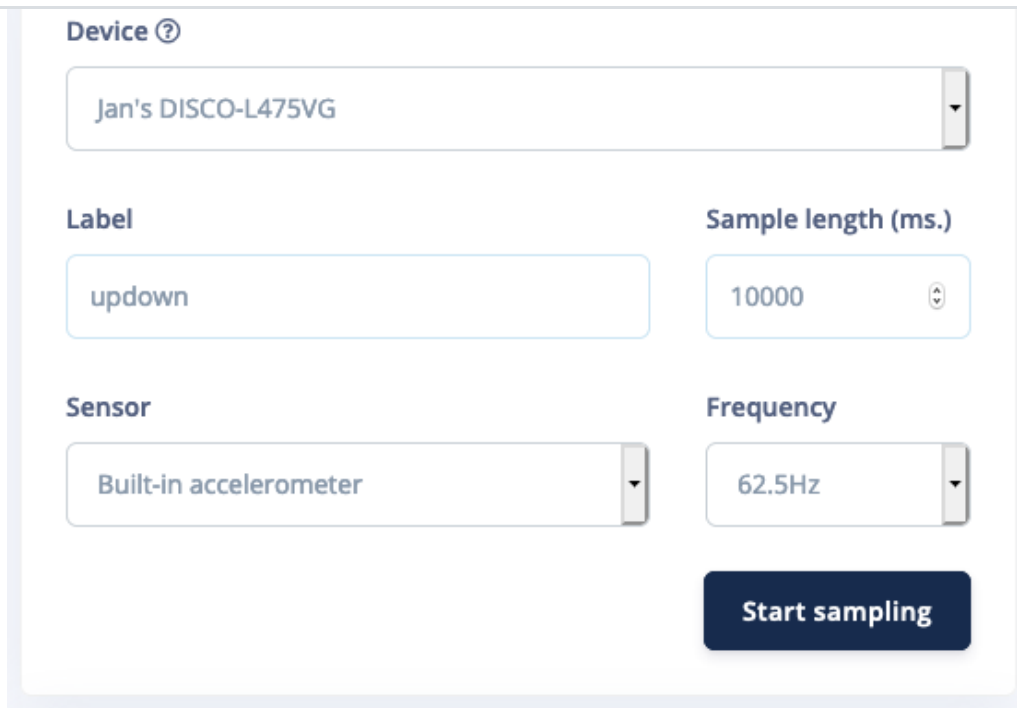
Device compatibility

Edge Impulse can ingest data from any device - including embedded devices that you already have in production. See the documentation for the [Ingestion service](#) for more information.

2. Collecting your first data

With your device connected we can collect some data. In the studio go to the **Data acquisition** tab. This is the place where all your raw data is stored, and - if your device is connected to the remote management API - where you can start sampling new data.

Under **Record new data**, select your device, set the label to `updown` , the sample length to `10000` , the sensor to `Built-in accelerometer` and the frequency to `62.5Hz` . This indicates that you want to record data for 10 seconds, and label the recorded data as `updown` . You can later edit these labels if needed.



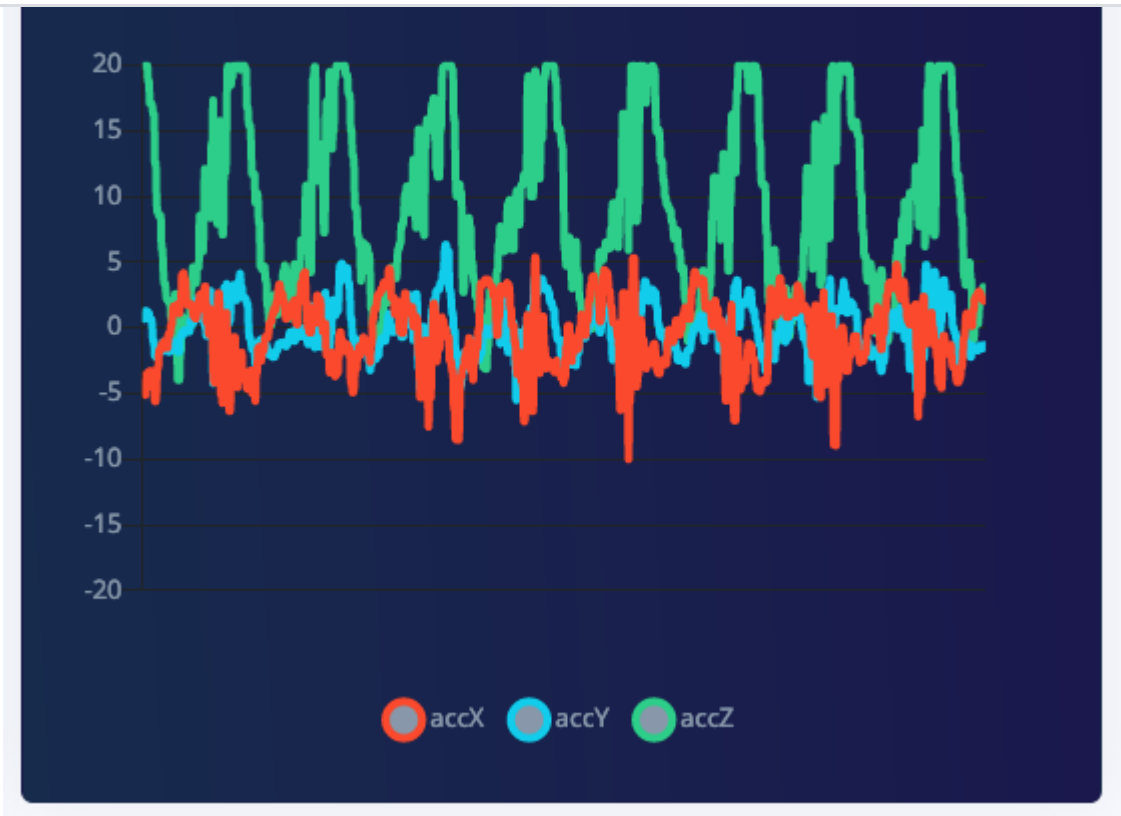
The image shows a 'Record new data' interface. It features a 'Device' dropdown menu with 'Jan's DISCO-L475VG' selected. Below this are four input fields: 'Label' with 'updown', 'Sample length (ms.)' with '10000', 'Sensor' with 'Built-in accelerometer', and 'Frequency' with '62.5Hz'. A dark blue 'Start sampling' button is positioned at the bottom right of the form.

Record new data screen.

After you click **Start sampling** move your device up and down in a continuous motion. In about twelve seconds the device should complete sampling and upload the file back to Edge Impulse. You see a new line appear under 'Collected data' in the studio. When you click it you now see the raw data graphed out. As the accelerometer on the development board has three axes you'll notice three different lines, one for each axis.

Continuous movement

It's important to do continuous movements as we'll later slice up the data in smaller windows.



Updown movement recorded from the accelerometer.

Machine learning works best with lots of data, so a single sample won't cut it. Now is the time to start building your own dataset. For example, use the following four classes, and record around 3 minutes of data per class:

- Idle - just sitting on your desk while you're working.
- Snake - moving the device over your desk as a snake.
- Wave - waving the device from left to right.
- Updown - moving the device up and down.

Variations

Make sure to perform variations on the motions. E.g. do both slow and fast movements and vary the orientation of the board. You'll never know how your user will use the device. It's best to collect samples of ~10 seconds each.

Prebuilt dataset

Alternatively, you can load an example test set that has about ten minutes of data in these classes (but how much fun is that?). See the [Continuous gestures dataset](#) for more information.

3. Designing an impulse

With the training set in place you can design an impulse. An impulse takes the raw data, slices it up in smaller windows, uses signal processing blocks to extract features, and then uses a learning block to classify new data. Signal processing blocks always return the same values for the same input and are used to make raw data easier to process, while learning blocks learn from past experiences.

In the studio go to **Create impulse**, set the window size to 2000 (you can click on the 2000 ms. text to enter an exact value), the window increase to 80, and add the 'Spectral Analysis' and 'Neural Network (Keras)' blocks. Then click **Save impulse**.

The screenshot shows the 'Create impulse' interface with four main blocks:

- Raw data** (red): Contains 'Axes' (accX, accY, accZ), 'Window size' (2000 ms), and 'Window increase' (80 ms).
- Spectral Analysis** (white): Contains 'Name' (Spectral features), 'Input axes' (accX, accY, accZ), and 'Output features' (33).
- Neural Network (Keras)** (purple): Contains 'Name' (NN Classifier), 'Input features' (Spectral features), and 'Output features' (4 (idle, snake, updown, wave)).
- Output features** (green): Shows 4 (idle, snake, updown, wave).

A 'Save Impulse' button is located below the Output features block. Below the main blocks are two dashed boxes: 'Add a processing block' and 'Add a learning block'.

First impulse, with one processing block and one learning block.

Configuring the spectral analysis block

To configure your signal processing block, click **Spectral features** in the menu on the left. This will show you the raw data on top of the screen (you can select other files via the drop down menu), and the results of the signal processing through graphs on the right. For the spectral features block you'll see the following graphs:

- After filter - the signal after applying a low-pass filter. This will remove noise.
- Frequency domain - the frequency at which signal is repeating (e.g. making one wave movement per second will show a peak at 1 Hz).
- Spectral power - the amount of power that went into the signal at each frequency.

A good signal processing block will yield similar results for similar data. If you move the sliding window (on the raw data graph) around, the graphs should remain similar. Also, when you switch to another file with the same label, you should see similar graphs, even if the orientation of the device was different.

Bonus exercise: filters

Try to reason about the filter parameters. What does the cut-off frequency control? And what do you see if you switch from a low-pass to a high-pass filter?

Once you're happy with the result, click **Save parameters**. This will send you to the 'Feature generation' screen. In here you'll:

Afterwards the 'Feature explorer' will load. This is a plot of all the extracted features against all the generated windows. You can use this graph to compare your complete data set. E.g. by plotting the height of the first peak on the X-axis against the spectral power between 0.5 Hz and 1 Hz on the Y-axis. A good rule of thumb is that if you can visually separate the data on a number of axes, then the machine learning model will be able to do so as well.



Examining your full dataset in the feature explorer.

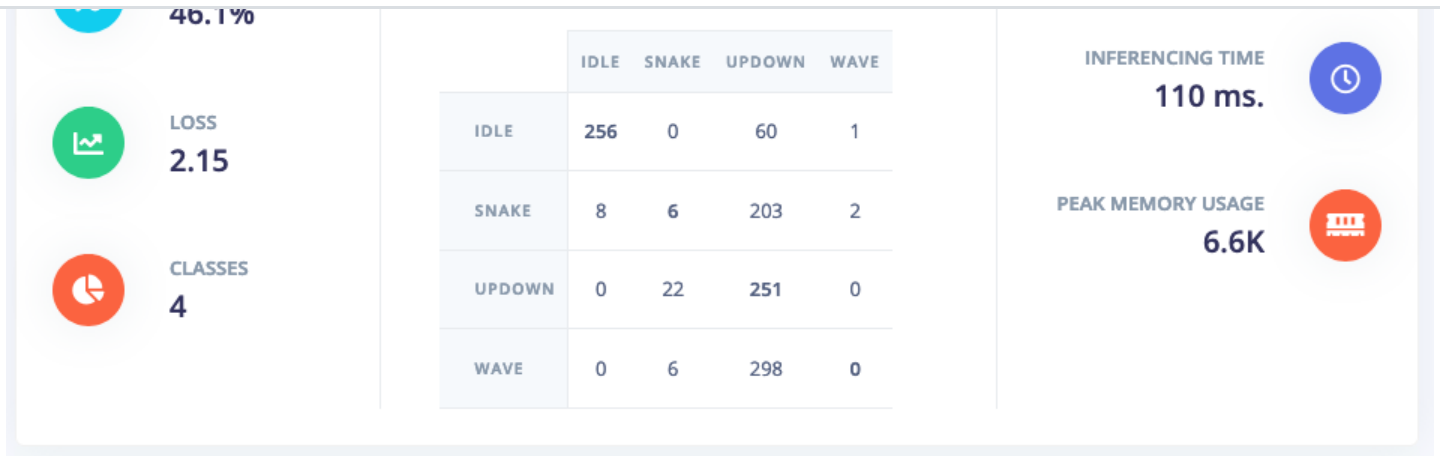
Configuring the neural network

With all data processed it's time to start training a neural network. Neural networks are a set of algorithms, modeled loosely after the human brain, that are designed to recognize patterns. The network that we're training here will take the signal processing data as an input, and try to map this to one of the four classes.

So how does a neural network know what to predict? A neural network consists of layers of neurons, all interconnected, and each connection has a weight. One such neuron in the input layer would be the height of the first peak of the X-axis (from the signal processing block); and one such neuron in the output layer would be wave (one the classes). When defining the neural network all these connections are initialized randomly, and thus the neural network will make random predictions. During training we then take all the raw data, ask the network to make a prediction, and then make tiny alterations to the weights depending on the outcome (this is why labeling raw data is important).

This way, after a lot of iterations, the neural network learns; and will eventually become much better at predicting new data. Let's try this out by clicking on **NN Classifier** in the menu.

Set 'Number of training cycles' to 1 .. This will limit training to a single iteration. And then click **Start training**.



Training performance after a single iteration. On the left a summary of the accuracy of the network, and in the middle a confusion matrix. This matrix shows when the network made correct and incorrect decisions. You see that idle is relatively easy to predict. Why do you think this is?

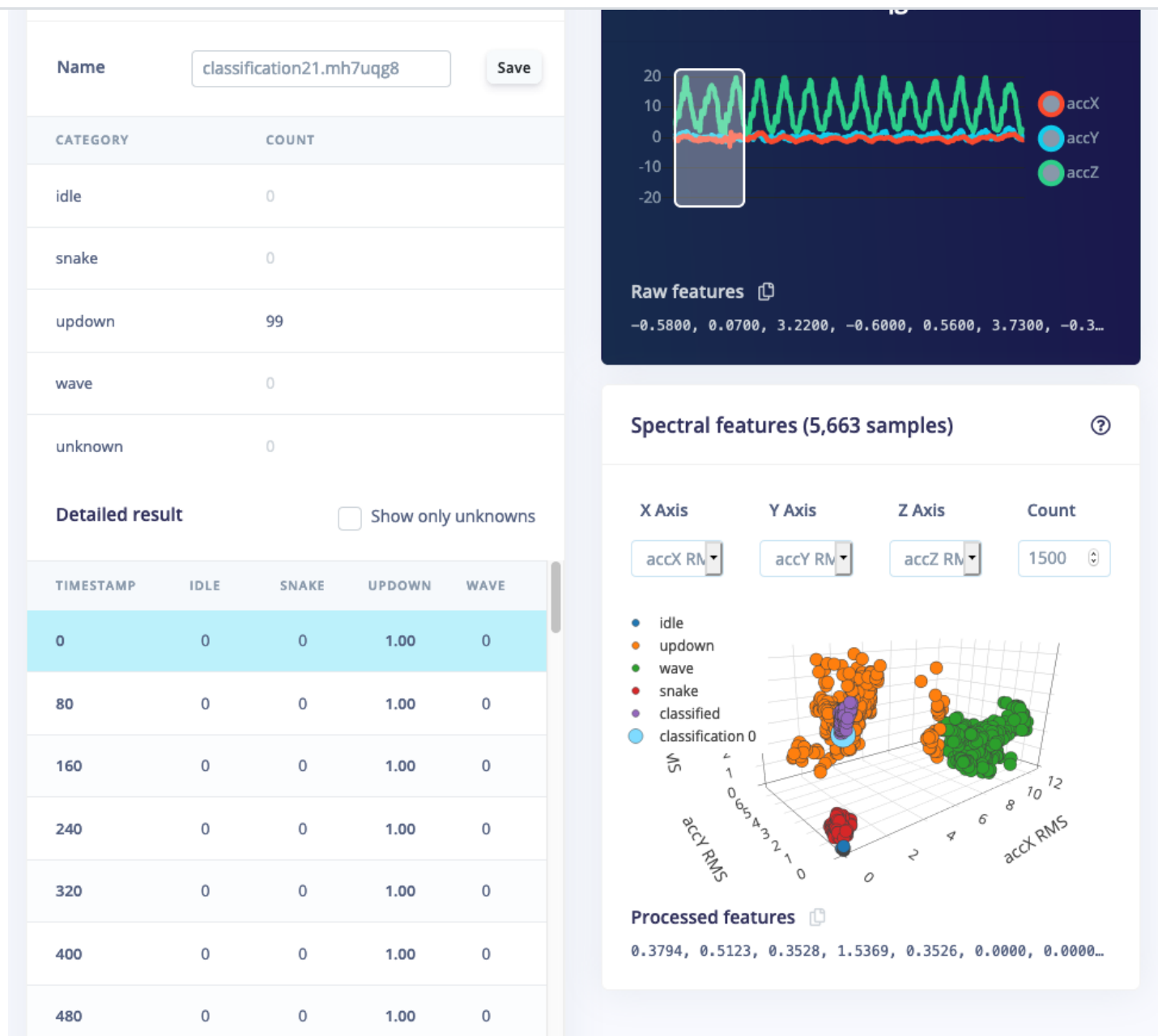
Now change 'Number of training cycles' to 2 and you'll see performance go up. Finally, change 'Number of training cycles' to 100 and let training finish. You've just trained your first neural network!

100% accuracy

You might end up with a 100% accuracy after training for 100 training cycles. This is not necessarily a good thing, as it might be a sign that the neural network is too tuned for the specific test set and might perform poorly on new data (overfitting). The best way to reduce this is by adding more data or reducing the learning rate.

4. Classifying new data

From the statistics in the previous step we know that the model works against our training data, but how well would the network perform on new data? Click on **Live classification** in the menu to find out. Your device should (just like in step 2) show as online under 'Classify new data'. Set the 'Sample length' to 5000 (5 seconds), click **Start sampling** and start doing movements. Afterwards you'll get a full report on what the network thought that you did.



Classification result. Showing the conclusions, the raw data and processed features in one overview.

If the network performed great, fantastic! But what if it performed poorly? There could be a variety of reasons, but the most common ones are:

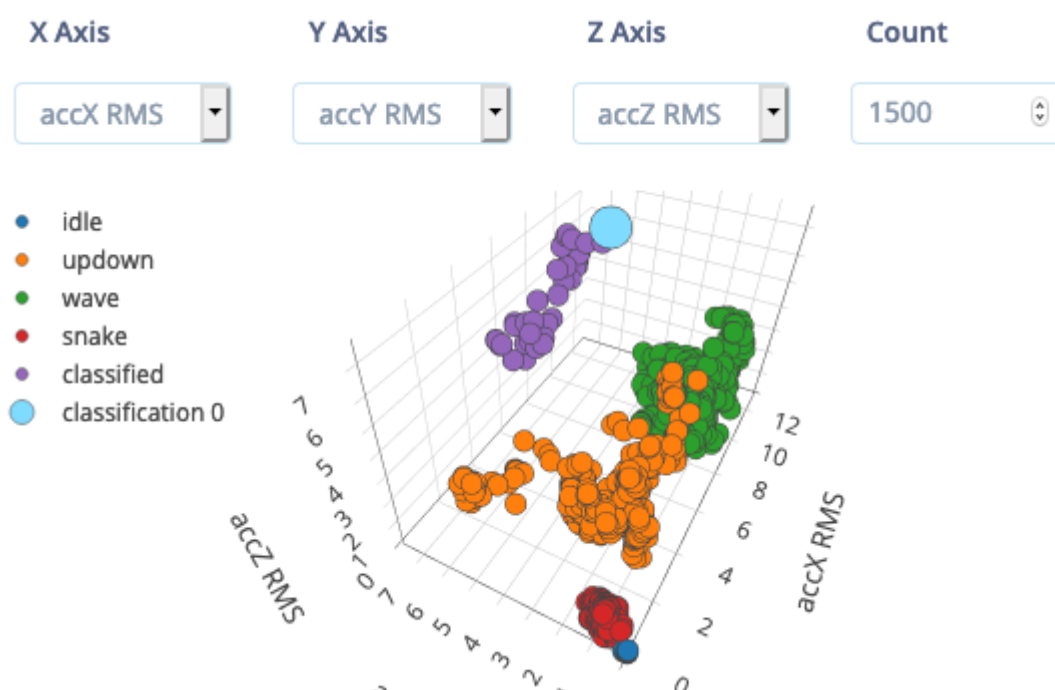
1. There is not enough data. Neural networks need to learn patterns in data sets, and the more data the better.
2. The data does not look like other data the network has seen before. This is common when someone uses the device in a way that you didn't add to the test set. You can add the current file to the test set by clicking **+**, then selecting **Move to training set**. Make sure to update the label under 'Data acquisition' before training.
3. The model has not been trained enough. Up the number of epochs to 200 and see if performance increases (the classified file is stored, and you can load it through 'Classify existing validation sample').
4. The model is overfitting and thus performs poorly on new data. Try reducing the learning rate or add more data.
5. The neural network architecture is not a great fit for your data. Play with the number of layers and neurons and see if performance improves.

As you see there is still a lot of trial and error when building neural networks, but we hope the visualizations help a lot. You can also run the network against the complete validation set through 'Model validation'. Think of the model

Neural networks are great, but they have one big flaw. They're terrible at dealing with data they have never seen before (like a new gesture). Neural networks cannot judge this, as they are only aware of the training data. If you give it something unlike anything it has seen before it'll still classify as one of the four classes.

Let's look at how this works in practice. Go to 'Live classification' and record some new data, but now vividly shake your device. Take a look and see how the network will predict something regardless.

So... how can we do better? If you look at the feature explorer on the accX RMS, accY RMS and accZ RMS axes, you should be able to visually separate the classified data from the training data. We can use this to our advantage by training a new (second) network that creates clusters around data that we have seen before, and compares incoming data against these clusters. If the distance from a cluster is too large you can flag the sample as an anomaly, and not trust the neural network.



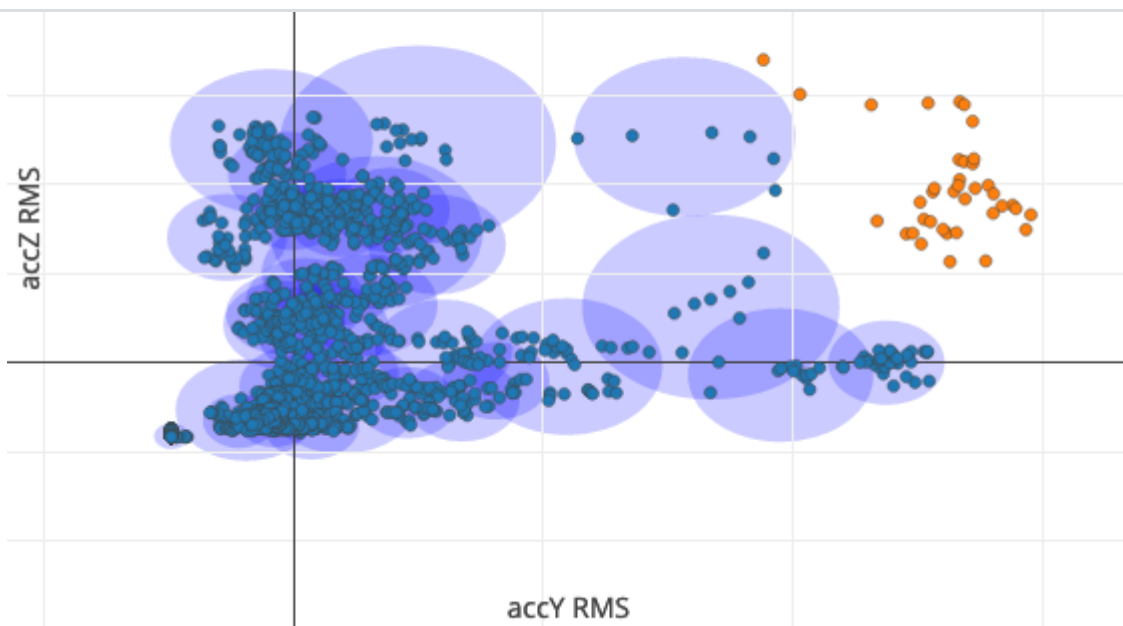
Shake data is easily separated from the training data.

To add this block go to **Create impulse**, click **Add learning block**, and select 'K-means Anomaly Detection'. Then click **Save impulse**.

To configure the clustering model click on **Anomaly detection** in the menu. Here we need to specify:

- The number of clusters. Here use 32.
- The axes that we want to select during clustering. As we can visually separate the data on the accX RMS, accY RMS and accZ RMS axes, select those.

Click **Start training** to generate the clusters. You can load existing validation samples into the anomaly explorer with the dropdown menu.



Known clusters in blue, the shake data in orange. It's clearly outside of any known clusters and can thus be tagged as an anomaly.

Axes

The anomaly explorer only plots two axes at the same time. Under 'average axis distance' you see how far away from each axis the validation sample is. Use the dropdown menu's to change axes.

If you now go back to 'Live classification' and load your last sample, it should now have tagged everything as anomaly. This is a great example where signal processing (to extract features), neural networks (for classification) and clustering algorithms (for anomaly detection) can work together.

6. Deploying back to device

With the impulse designed, trained and verified you can deploy this model back to your device. This makes the model run without an internet connection, minimizes latency, and runs with minimum power consumption. Edge Impulse can package up the complete impulse - including the signal processing code, neural network weights, and classification code - up in a single C++ library that you can include in your embedded software.

Mobile phone

Your mobile phone can build and download the compiled impulse directly from the mobile client. See 'Deploying back to device' on the [Using your mobile phone](#) page.

To export your model, click on **Deployment** in the menu. Then under 'Build firmware' select your development board, and click **Build**. This will export the impulse, and build a binary that will run on your development board in a single step. After building is completed you'll get prompted to download a binary. Save this on your computer.

Flashing the device

We can connect to the board's newly flashed firmware over serial. Open a terminal and run:

```
$ edge-impulse-run-impulse
```

Serial daemon

If the device is not connected over WiFi, but instead connected via the Edge Impulse serial daemon, you'll need stop the daemon. Only one application can connect to the development board at a time.

This will sample data from the sensor, run the signal processing code, and then classify the data:

```
Starting inferencing in 2 seconds...
Sampling... Storing in file name: /fs/device-classification261
Tensor shape: 4
Predictions (DSP: 17 ms., Classification: 1 ms., Anomaly: 0 ms.):
  idle: 0.00004
  snake: 0.00012
  updown: 0.00009
  wave: 0.99976
  anomaly score: 0.032
Finished inferencing, raw data is stored in '/fs/device-classification261'. Use AT+UPLOADFILE to send back
```

Continuous movement

We trained a model to detect continuous movement in 2 second intervals. Thus, changing your movement while sampling will yield incorrect results. Make sure you've started your movement when 'Sampling...' gets printed. In between sampling you have two seconds to switch movements.

Victory! You've now built your first on-device machine learning model.



7. Conclusion

Congratulations! you've used Edge Impulse to train a machine learning model capable of recognizing your gestures and understand how you can build models that classify sensor data or find anomalies. Now that you've trained your model you can integrate your impulse in the firmware of your own embedded device, see [Running your impulse locally](#). There are examples for Mbed OS, Arduino, STM32CubeIDE, Eta Compute, and any other target that supports a C++ compiler.

Or if you're interested in more, see our tutorials on [Recognize sounds from audio](#) or [Adding sight to your sensors](#). If you have a great idea for a different project, that's fine too. Edge Impulse lets you capture data from any sensor, build [custom processing blocks](#) to extract features, and you have full flexibility in your Machine Learning pipeline with the learning blocks.

We can't wait to see what you'll build! 🚀